



PROGRAMAÇÃO ORIENTADA A OBJETOS

ESCOLHA VIVER
MAIS EXPERIÊNCIAS.

Flavio Viola
flavio.viola@uva.br

UVA ●
A sua melhor escolha.



APRESENTAÇÃO PROFESSOR X ALUNO

ESCOLHA VIVER
MAIS EXPERIÊNCIAS.

Currículo 2022
SEMESTRAL

UVA 
A sua melhor escolha.

Apresentação do Professor



Flavio Viola

email: flavio.viola@uva.br

- ✓ **Doutorado em Engenharia Mecânica (COPPE/UFRJ);**
- ✓ **Mestrado (UFF) e Graduação em Ciência da Computação (FIAA);**
- ✓ **Especialização em Análise e Projeto de Desenvolvimento de Sistemas (PUC-RJ).**



EQUIPES NO TEAMS

ESCOLHA VIVER
MAIS EXPERIÊNCIAS.

UVA 

A sua melhor escolha.

Nome da equipe no Teams:

POO_2026_2_VIOLA

Código equipe no Teams:

gs24vyr

Nome da equipe no Teams:

POO_VIOLA_TIJUCA_2026_2

Código equipe no Teams:

afco3pb



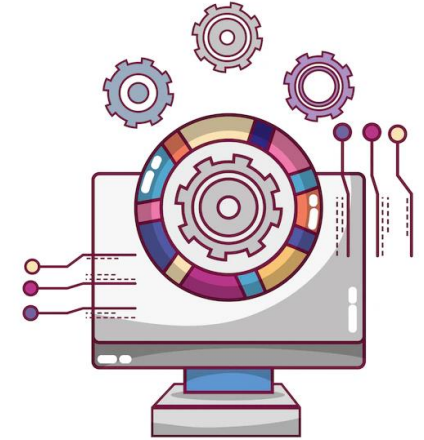
COMPETÊNCIAS E HABILIDADES OBJETIVO DA DISCIPLINA

ESCOLHA VIVER
MAIS EXPERIÊNCIAS.

UVA ●
A sua melhor escolha.

COMPETÊNCIAS E HABILIDADES:

As competências gerais desenvolvidas ao longo do curso envolvem capacidade de abstração, senso crítico, raciocínio lógico e matemático, iniciativa pessoal, tolerância, flexibilidade, dinamismo, comunicação oral e escrita, capacidade de aplicar conhecimentos de forma independente e capacidade de antecipar e solucionar problemas.



OBJETIVO DA DISCIPLINA:

- Ao final desta Disciplina, espera-se que o aluno tenha aprendido a: Criar programas na linguagem Java utilizando conceitos básicos e avançados de Programação Orientada a Objetos.



REFERÊNCIAS BIBLIOGRÁFICAS

ESCOLHA VIVER
MAIS EXPERIÊNCIAS.

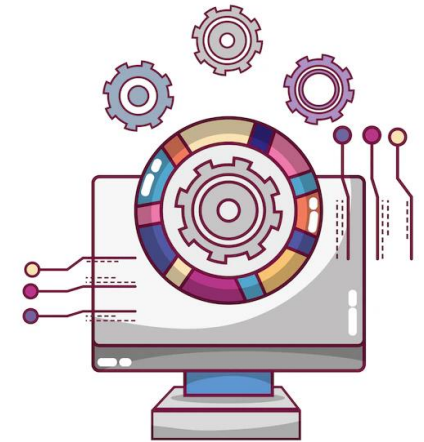
UVA 

A sua melhor escolha.

BIBLIOGRAFIA RECOMENDADA:

Referências Básicas:

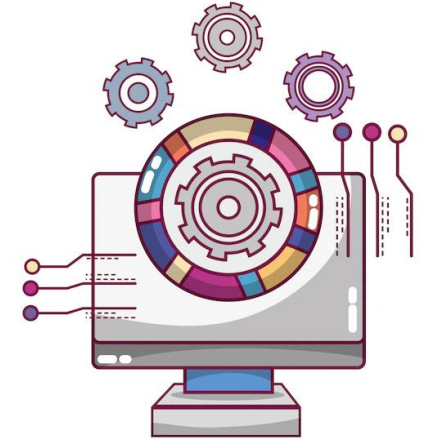
- DEITEL, Paul; DEITEL, Harvey. **Java: Como programar**. 10ª ed., Pearson, 2017.
- FURGERI, Sérgio. **Java 8 - Ensino Didático**. Érica, 2015.
- SCHILDT, Herbert. **Java para Iniciantes**. 6ª ed., Bookman, 2015.



BIBLIOGRAFIA RECOMENDADA:

Referências Complementares:

- ARNOLD, Ken; GOSLING, James; HOLMES, David. **A Linguagem de Programação Java**. 4ª ed., Bookman, 2007.
- HORSTMANN, Cay. **Conceitos de Computação com Java**, 5ª ed., ed. Bookman, 2009.
- WINDER, Russel; ROBERTS, Graham. **Desenvolvendo Software em Java**. 3ª ed., LTC, 2009.
- HORSTMANN, Cay; CORNELL, Gary. **Core Java Volume 1: Fundamentos**, 8ª ed., Pearson, 2010.





PLANEJAMENTO DAS AULAS

ESCOLHA VIVER
MAIS EXPERIÊNCIAS.

UVA ●

A sua melhor escolha.

PLANEJAMENTO DAS AULAS:



Unidade 1 – Fundamentos da Linguagem Java

Objetivos

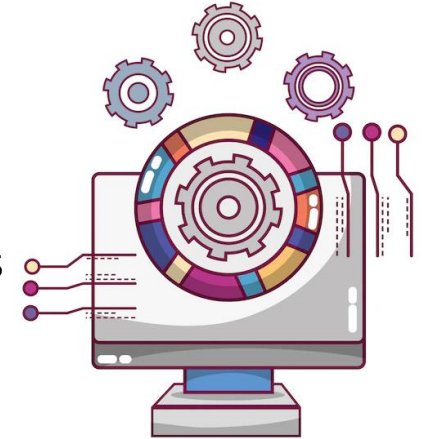
Criar programas básicos na linguagem Java usando tipos primitivos de dados e estruturas de controle.

Conteúdos

1.1 Paradigma estruturado x paradigma OO. Histórico e características da linguagem. Estrutura básica de um programa Java. Tipos primitivos de dados. Declaração e inicialização de constantes e variáveis.

1.2 Entrada e saída de dados (modo texto). Operadores e expressões aritméticas. Operadores de atribuição aritmética. Operadores relacionais e lógicos. Operador condicional (? :). Conversão de tipos (*type casting*).

1.3 Comandos de seleção (*if..else, switch*). Comandos de repetição (*while, do..while, for*). Comandos de desvio de fluxo (*break, continue*).



PLANEJAMENTO DAS AULAS:



Unidade 2 – Classes, Atributos e Objetos

Objetivos

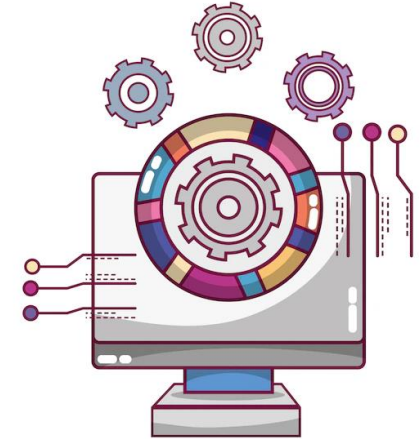
Criar programas na linguagem Java que implementem a definição de classes com atributos e a criação de objetos básicos.

Conteúdos

2.1 Conceitos de Orientação a Objetos: objetos no mundo real; características (atributos) e comportamentos (métodos) dos objetos; classificação, encapsulamento e abstração.

2.2 Classes e atributos: definição e sintaxe. Visibilidade de classes e atributos em Java.

2.3 Construtores: definição e sintaxe. Criação de objetos com operador `new`. Referência *this*.
Sobrecarga de construtores.



PLANEJAMENTO DAS AULAS:



Unidade 3 – Métodos

Objetivos

Criar programas na linguagem Java que implementem a definição e a chamada de métodos de acesso, de instância e estáticos.

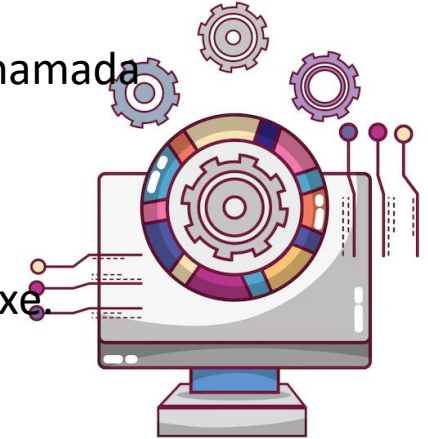
Conteúdos

3.1 Princípio do Encapsulamento. Métodos de acesso: definição e sintaxe.

3.2 Métodos de instância: definição e sintaxe. Chamada de métodos a partir de objetos.

Sobrecarga de métodos. Passagem de parâmetros nos métodos: tipos primitivos e objetos.

3.3 Métodos estáticos e atributos estáticos: definição e sintaxe. Classe *Math*.



PLANEJAMENTO DAS AULAS:



Unidade 4 – Strings e Vetores

Objetivos

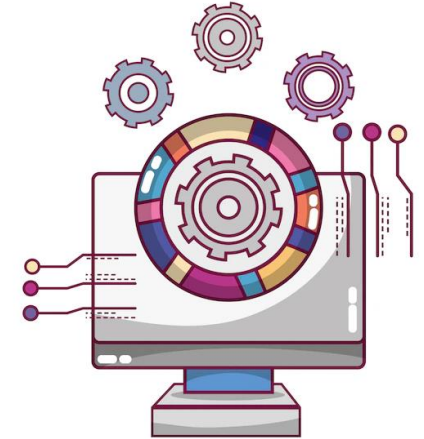
Criar programas na linguagem Java que manipulem cadeias de caracteres e vetores de tipos primitivos de dados.

Conteúdos

4.1 Cadeias de caracteres com a classe *String*. Métodos básicos da classe *String*.

4.2 Comparação de strings: métodos *equals* e *compareTo*.

4.3 Vetores de tipos primitivos: definição e sintaxe. Criação de vetores com *new*. Inicialização de vetores. Atributo *length*. Percorrendo vetores com *for* e *for each*.



PLANEJAMENTO DAS AULAS:



Unidade 5 – Relacionamentos entre Objetos

Objetivos

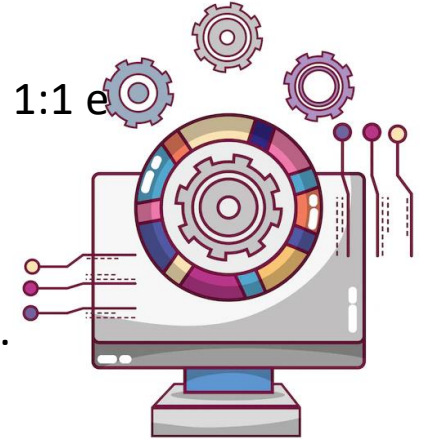
Criar programas na linguagem Java que implementem relacionamentos 1:1 e 1:N usando vetores de objetos.

Conteúdos

5.1 Vetores de objetos. Chamada de métodos para os objetos do vetor.

5.2 Relacionamentos entre objetos: associação, agregação e composição. Multiplicidade de relacionamentos: 1:1, 1:N e M:N. Implementação de relacionamentos 1:1.

5.3 Implementação de relacionamentos 1:N com vetor de objetos.



PLANEJAMENTO DAS AULAS:



Unidade 6 – Herança e Polimorfismo

Objetivos

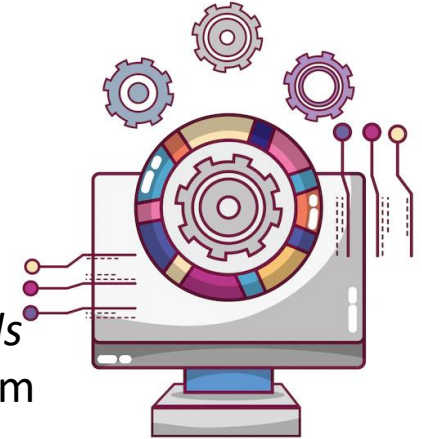
Criar programas na linguagem Java que implementem hierarquia de classes e polimorfismo.

Conteúdos

6.1 Conceito de Herança. Herança simples e múltipla. Comando *extends* e herança de atributos e métodos em Java. Chamada de construtores em uma hierarquia. Referência *super*.

6.2 Conceito de polimorfismo. Sobrescrita de métodos em uma hierarquia. Execução de métodos polimórficos. Referência *super*.

6.3 Classes abstratas e métodos abstratos. Operador *instanceof*.



PLANEJAMENTO DAS AULAS:



Unidade 7 – Tratamento de Exceção

Objetivos

Criar programas em Java que implementem os mecanismos de tratamento e lançamento de exceções.

Conteúdos

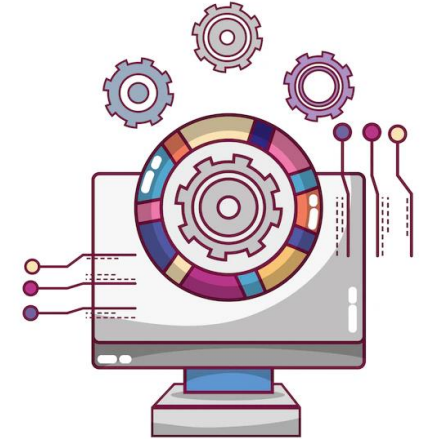
7.1 Conceito de Exceção e Erro. Hierarquia de exceções no Java.

Exceções

verificadas e não verificadas.

7.2 Tratamento de exceções com o comando *try..catch*. Comando *throws*.

7.3 Criação e lançamento de exceções com o comando *throw*.



PLANEJAMENTO DAS AULAS:



Unidade 8 – Tópicos Avançados em Programação OO

Objetivos

Ao final desta Unidade, espera-se que o aluno tenha aprendido a:
Criar programas na linguagem Java que manipulem objetos *wrapper* e coleções de objetos com *ArrayList*.

Conteúdos

- 8.1 Classe *Object*. Sobrecarga dos métodos *equals* e *toString*. Classes *wrapper*.
- 8.2 Definição de Interface. Definição de coleções. Hierarquia de coleções: *set*, *list* e *map*.
- 8.3 *ArrayList*: definição e criação de listas. Métodos básicos para inclusão, alteração e remoção de elementos. Listas tipadas.



PLANEJAMENTO DAS AULAS:

PLANEJAMENTO DAS AULAS	
DATA	CONTEÚDO
07/ago	1.1
14/ago	1.2 e 1.3
21/ago	1.3 , 2.1 e 2.2
28/ago	2.2 e 2.3
04/set	3.1 e 3.2
11/set	3.2 e 3.3
18/set	4.1 e 4.2
25/set	A1
02/out	Revisão A1 e 4.3
09/out	5.1 e 5.2
16/out	5.3 e 6.1
23/out	6.1 e 6.2
30/out	6.3 e 7.1
06/nov	7.2 e 7.3
13/nov	8.1 , 8.2 e 8.3
20/nov	FERIADO
27/nov	A2
04/dez	Revisão A2
11/dez	A3



CRITÉRIOS DE AVALIAÇÃO E APROVAÇÃO

ESCOLHA VIVER
MAIS EXPERIÊNCIAS.

UVA 

A sua melhor escolha.

CRITÉRIOS DE AVALIAÇÃO:



A1 – 25/09 - Prova individual... 10,0

A2 – 27/11- Prova individual e trabalhos... 10,0

A3 – 11/12 - Prova individual... 10,0

CRITÉRIOS DE APROVAÇÃO:

Média $\geq 6,0$ e mínimo de 75% de presença

Então, o aluno estará **APROVADO. Nota de A1 ou A2 inferior a 5,0 será zerada no sistema.**

Observação: A A3 pode ser 2ª chamada para a A1 ou A2, ou pode substituir a menor nota obtida.

Semana 01

**Apresentação da disciplina e plano de aula;
Unidade 1 – Fundamentos da Linguagem Java**

Programa

Programação Orientada a Objetos

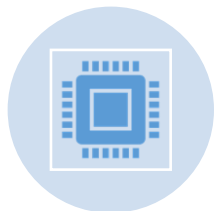
1.1 Paradigma estruturado x paradigma OO. Histórico e características da linguagem. Estrutura básica de um programa Java. Tipos primitivos de dados. Declaração e inicialização de constantes e variáveis.

1.2 Entrada e saída de dados (modo texto). Operadores e expressões aritméticas. Operadores de atribuição aritmética. Operadores relacionais e lógicos. Operador condicional (? :). Conversão de tipos (type casting).

1.3 Comandos de seleção (if..else, switch). Comandos de repetição (while, do..while, for). Comandos de desvio de fluxo (break, continue).

Introdução ao Java

Programação OO



Linguagem de programação orientada a objetos desenvolvida por um time de programadores liderados por Patrick Naughton, Mike Sheridan e James Gosling na empresa Sun Microsystems.



É uma linguagem que é compilada para um “*bytecode*” que é interpretada por uma máquina virtual – JVM (*Java virtual machine*).



Empresa fabricante de computadores, semicondutores e softwares com sede em Santa Clara, Califórnia – EUA. Em abril de 2009, a Oracle Corporation comprou a Sun Microsystems, transformando a indústria de Tecnologia da Informação.



É a compilação de um programa escrito em Java com o objetivo de ser interpretado pela máquina virtual Java - JVM

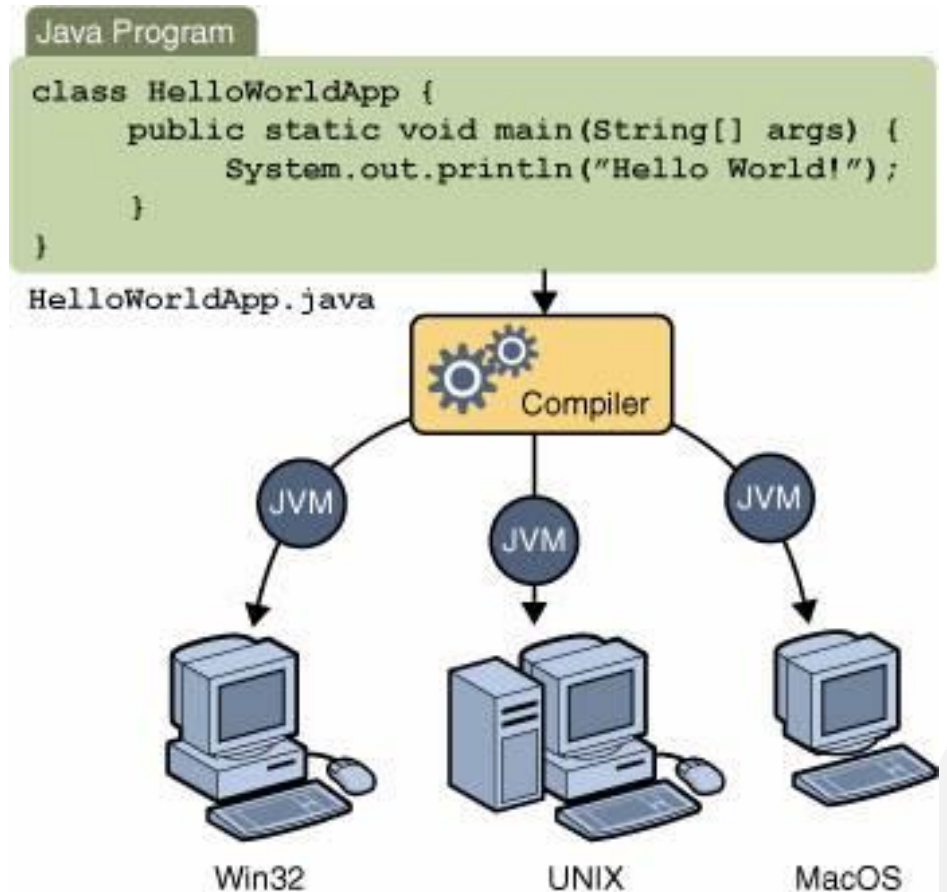


Java Virtual Machine – é um programa carrega e interpreta os aplicativos Java, convertendo os *bytecodes* em códigos executáveis de máquina.

Principais Características

Portabilidade

- o Java pode ser executado em qualquer plataforma ou equipamento que possua um interpretador Java, e que tenha sido especialmente compilado para o sistema a ser utilizado



Principais Características

Orientada a Objetos

- Orientada a Objetos – contém a sintaxe similar a linguagem C/C++, mas é baseada no modelo Simula67.
- Segurança – Pode ser executado via rede com restrições de execução, além de proteger o sistema do cliente contra possíveis ataques não-intencionais.

Simula 67 é uma [linguagem de programação](#), extensão da [ALGOL 60](#), projetada para apoiar a [Simulação de Eventos Discretos](#), criadas entre [1962](#) e [1968](#) por [Kristen Nygaard](#) e [Ole-Johan Dahl](#) no [Centro Norueguês de Computação](#) em [Oslo](#).

O impacto da Simula 67 foi enorme, devido a ser a primeira linguagem [orientada a objetos](#), porém levou tempo para se concretizar. Na verdade, a [Orientação a objeto](#) teve que esperar a criação da [Smalltalk 80](#) para definir seu impacto.

Fonte: Wikipedia

Principais Características

Por que estudar Java

- Orientação a Objetos: Totalmente OO – permitindo herança e reutilização de código de forma dinâmica e estática.
- Facilidade: Derivada do C/C++ - Bem Familiar. O ambiente retira do programador a responsabilidade de gerenciar a memória e os ponteiros.

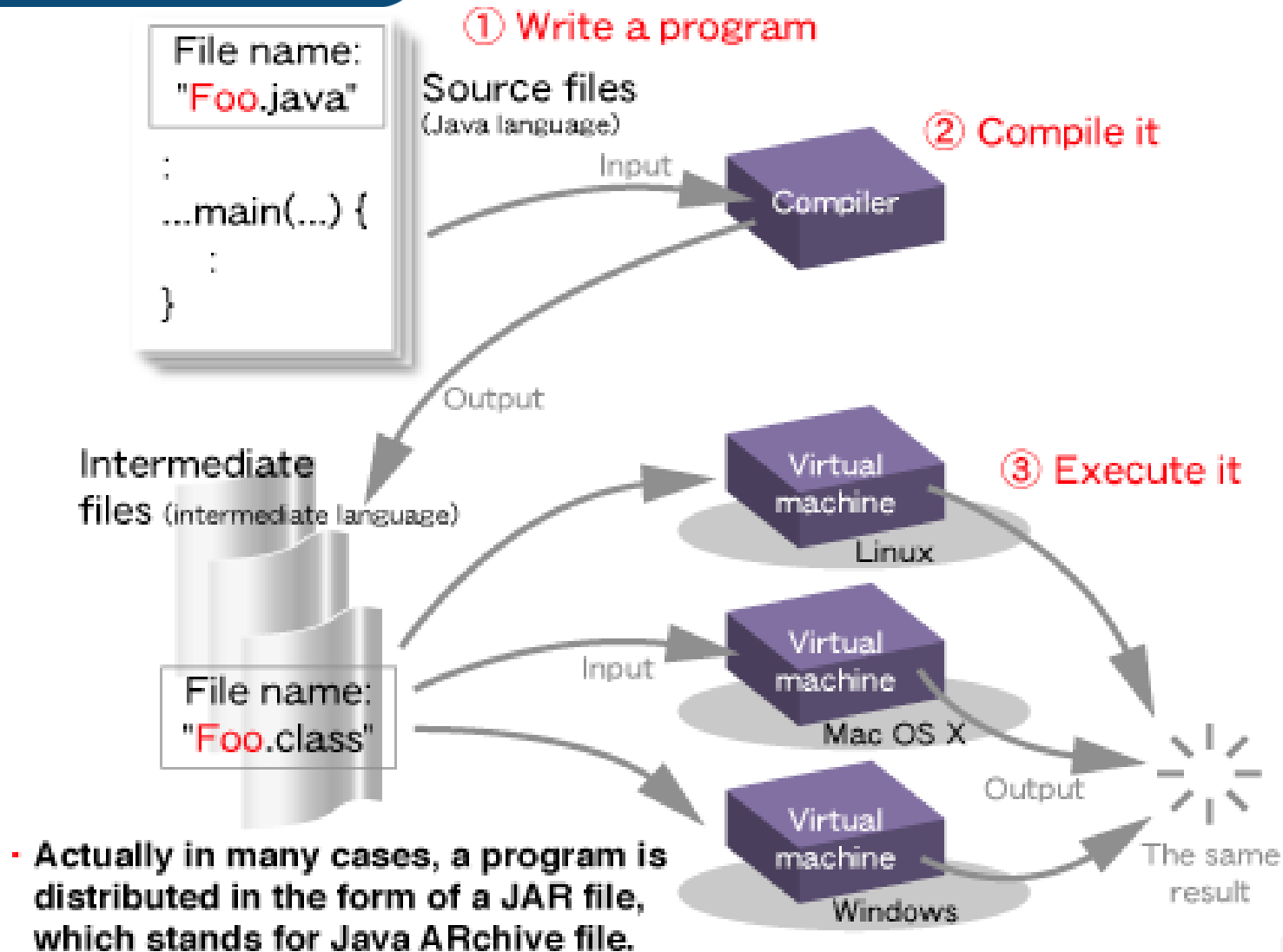
Utilizações e aplicações do Java

Por que estudar Java

- ❑ Desenvolvimento de aplicativos corporativos de grande porte;
- ❑ Desenvolvimento de aplicativos Web – *Front e Back end*;
- ❑ Fornecimento de aplicativos para dispositivos voltados para o consumo popular (celulares, pagers, PDAs, etc);
- ❑ Interligação com sistemas legados
- ❑ Muitos outros propósitos

Resumindo

Por que estudar Java

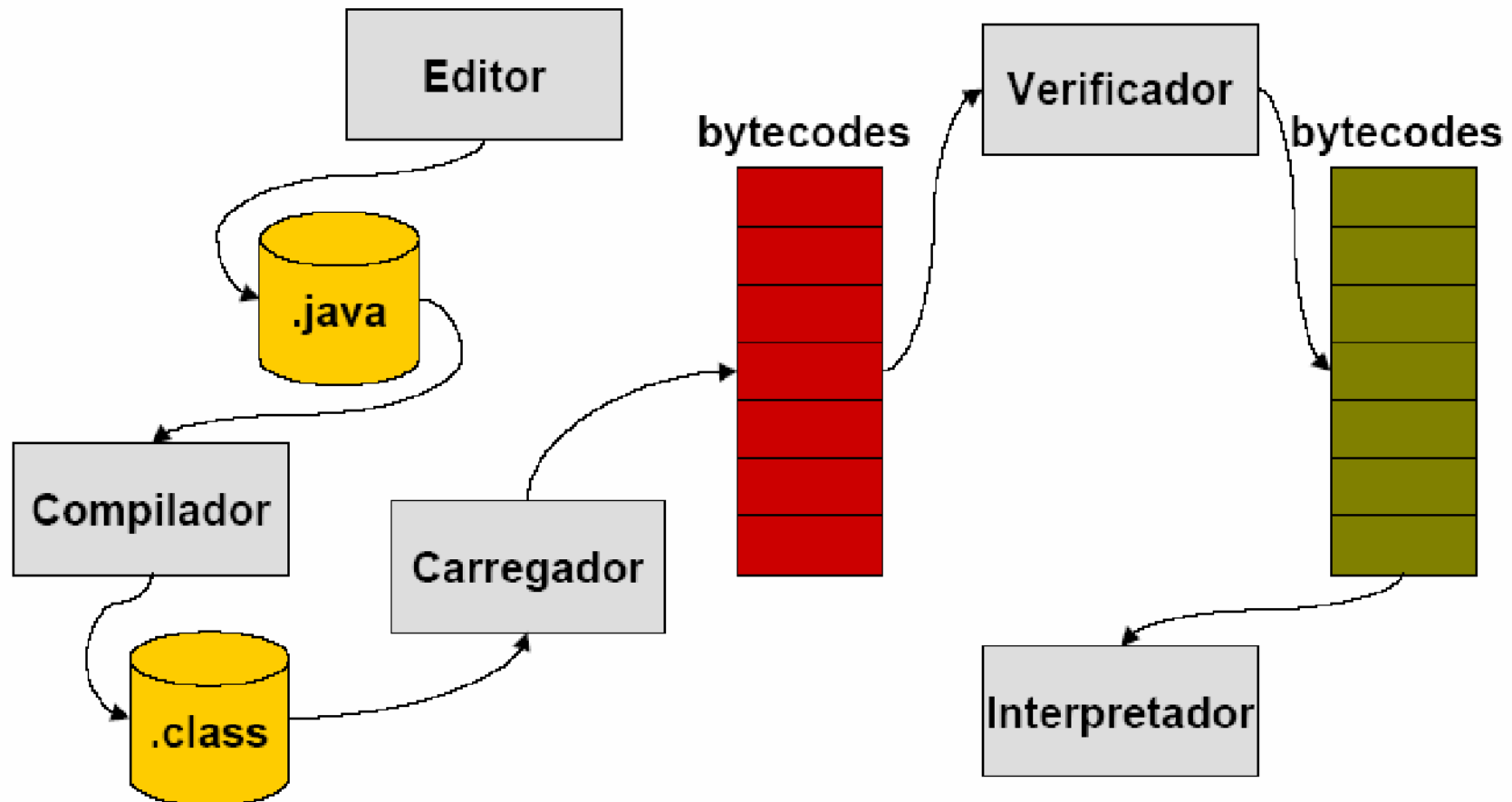


Em software, **JAR** (*Java **AR**chive*) é um arquivo/ficheiro compactado, usado para distribuir um conjunto de classes Java; um aplicativo Java; ou outros itens, como imagens, XMLs, entre outros. É usado para armazenar classes compiladas e metadados associados que podem constituir um programa.

Fonte: Wikipedia

Funcionamento de um programa

Por que estudar Java



Funcionamento de um programa

O que é Bytecode

O código de um [programa de computador](#) escrito na linguagem [Java](#) é [compilado](#) para uma forma intermediária de código denominada **[BYTECODE](#)**, que é interpretada pelas [Máquinas Virtuais Java](#) (JVMs).

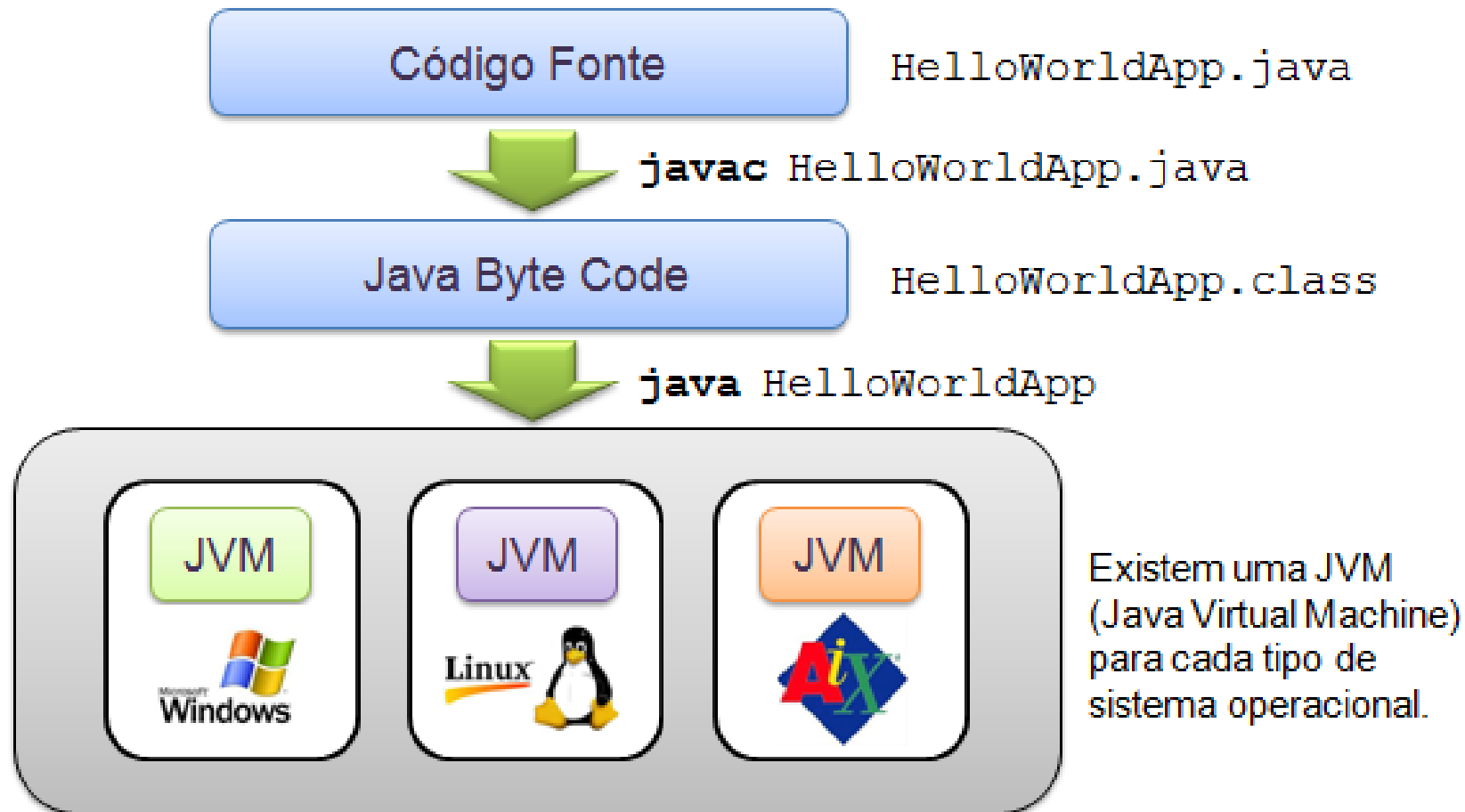
É essa característica que faz com que os programas Java sejam independentes de plataforma, executando em qualquer sistema que possua uma JVM.

Um programador Java não precisa entender — e nem tomar conhecimento — dos **bytecodes Java** para ser proficiente na linguagem, da mesma forma que um programador de qualquer linguagem de alto nível compilada para linguagem de máquina não precisa conhecer a linguagem de montagem do computador hospedeiro para escrever bons programas naquela linguagem.

Fonte: Wikipedia

Interoperabilidade

Por que estudar Java



Ambiente de programação

Por que estudar Java

- Desenvolvimento de aplicativos corporativos de grande porte;
- Java **SDK** (antigamente, JDK): conjunto de aplicativos que **permite realizar a compilação e a execução** de programas escritos na linguagem Java.
 - ✓ o compilador Java, javac,
 - ✓ o interpretador de aplicações Java (máquina virtual), java e
 - ✓ o interpretador de applets Java, appletviewer.
- Este ambiente pode ser baixado gratuitamente a partir do site da Sun Microsystems <http://java.sun.com>.

As siglas do Java

Programação OO

- J2SE – Java2 Standard Edition – desenvolvimento de aplicações na arquitetura Cliente – Servidor.
- J2EE – Java2 Enterprise Edition – Fornece um conjunto de APIs para o desenvolvimento corporativo e foca na integração entre sistemas. Disponibiliza alta distribuição de objetos e oferece total suporte a tecnologias para Internet.
- J2ME – Java2 Micro Edition – Fornece as APIs para o desenvolvimento de aplicações para computação móvel, em pequenos dispositivos ou tecnologias embarcadas.

Tipos de programas Java

Programação OO

- Stand – alone – aplicações baseadas na J2SE que tem total acesso aos recursos do sistema, memória, disco, rede, dispositivos, etc.
- Java applets – Pequenas aplicações; não têm acesso aos recursos hardware; necessita navegador com suporte J2SE
- Java servlets – executados em servidores Web, baseados na J2EE, usados para gerar conteúdos dinâmicos-websites.
- Java midlets – pequenos, extremamente seguros e construídos para serem executadas dentro do J2ME.
- JavaBeans – são componentes para serem manipulados visualmente

Montar Ambiente – Baixar JDK para instalação

Instalar um JDK (Java Development Kit)

- ✓ <https://www.oracle.com/br/java/technologies/downloads/#jdk17-windows>
- ✓ JDK Development Kit 17.0.8 downloads

JDK 21 JDK 17 GraalVM for JDK 21 GraalVM for JDK 17

JDK Development Kit 17.0.8 downloads

JDK 17 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions](#) (NFTC).

JDK 17 will receive updates under the NFTC, until September 2024. Subsequent JDK 17 updates will be licensed under the [Java SE OTN License](#) (OTN) and production use beyond the [limited free grants](#) of the OTN license will [require a fee](#).

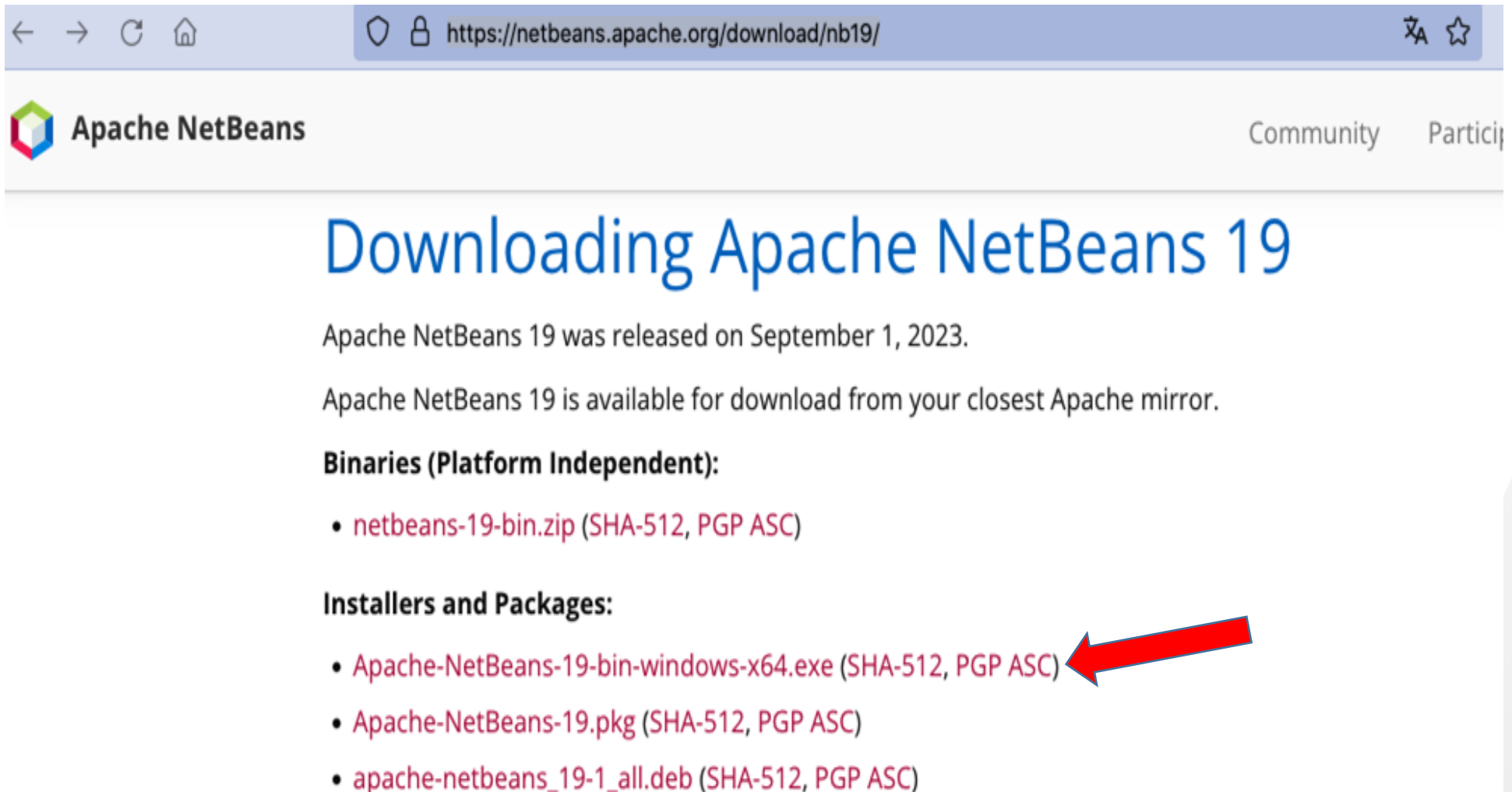
Linux macOS Windows

Product/file description	File size	Download
x64 Compressed Archive	172.38 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.zip (sha256)
x64 Installer	153.48 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.exe (sha256)
x64 MSI Installer	152.27 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.msi (sha256)

MONTAR O AMBIENTE: Baixando a IDE

Baixar o Netbeans 19

✓ <https://netbeans.apache.org/download/nb19/>



The screenshot shows the Apache NetBeans website's download page for version 19. The browser's address bar displays the URL <https://netbeans.apache.org/download/nb19/>. The page header includes the Apache NetBeans logo and navigation links for 'Community' and 'Participate'. The main heading is 'Downloading Apache NetBeans 19'. Below this, it states that Apache NetBeans 19 was released on September 1, 2023, and is available for download from the closest Apache mirror. The page lists two categories of downloads: 'Binaries (Platform Independent)' and 'Installers and Packages'. A red arrow points to the first item in the 'Installers and Packages' list: 'Apache-NetBeans-19-bin-windows-x64.exe (SHA-512, PGP ASC)'.

Apache NetBeans

Community Participate

Downloading Apache NetBeans 19

Apache NetBeans 19 was released on September 1, 2023.

Apache NetBeans 19 is available for download from your closest Apache mirror.

Binaries (Platform Independent):

- [netbeans-19-bin.zip](#) (SHA-512, PGP ASC)

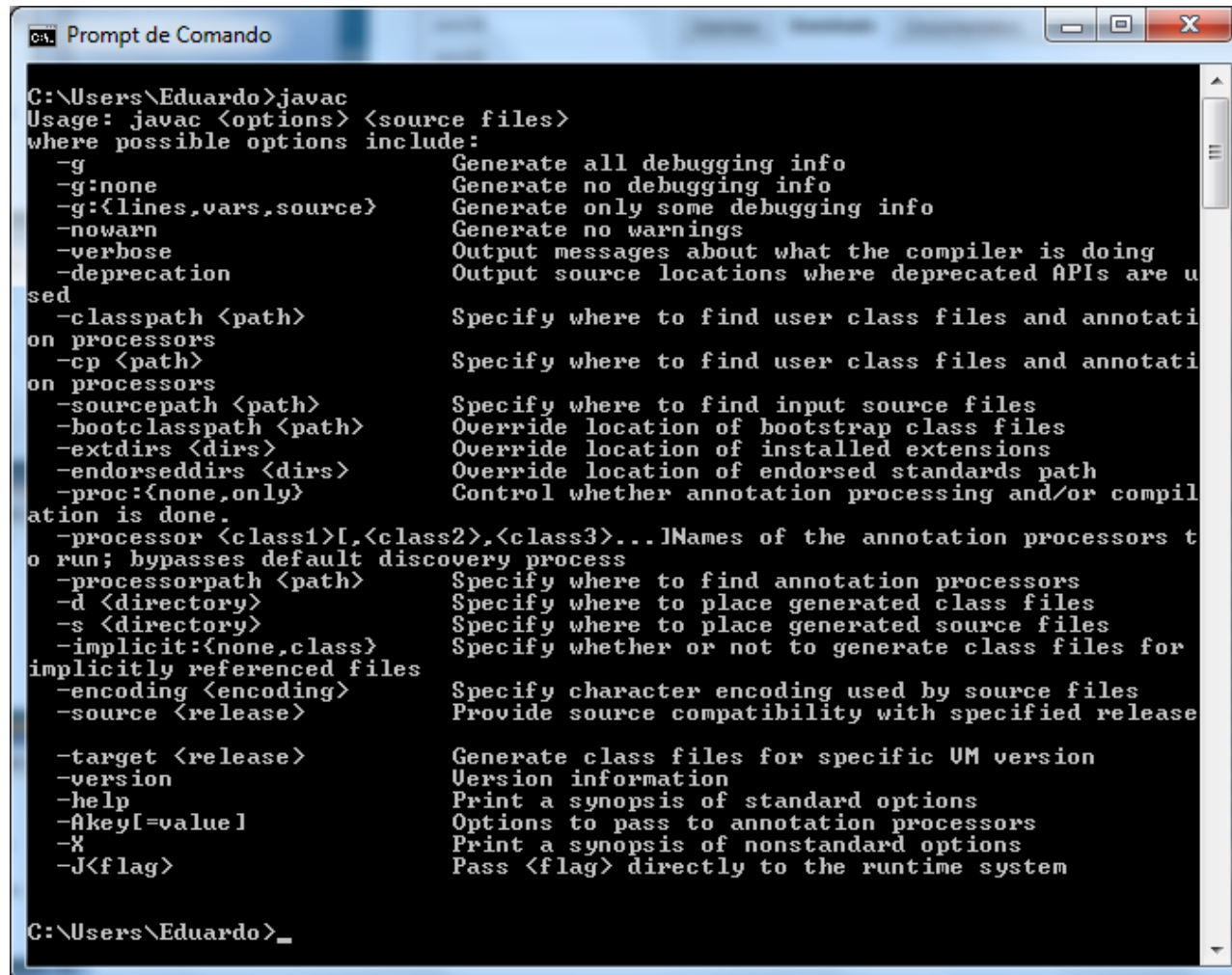
Installers and Packages:

- [Apache-NetBeans-19-bin-windows-x64.exe](#) (SHA-512, PGP ASC)
- [Apache-NetBeans-19.pkg](#) (SHA-512, PGP ASC)
- [apache-netbeans_19-1_all.deb](#) (SHA-512, PGP ASC)

Conferindo Instalação

Montando Ambiente

- Abra o Prompt de CMD
- Digite javac
- Aparecerá o help do comando compilador Java



```
C:\Users\Eduardo>javac
Usage: javac <options> <source files>
where possible options include:
  -g               Generate all debugging info
  -g:none          Generate no debugging info
  -g:{lines,vars,source}  Generate only some debugging info
  -nowarn          Generate no warnings
  -verbose         Output messages about what the compiler is doing
  -deprecation     Output source locations where deprecated APIs are used
  -classpath <path> Specify where to find user class files and annotation processors
  -cp <path>       Specify where to find user class files and annotation processors
  -sourcepath <path> Specify where to find input source files
  -bootclasspath <path> Override location of bootstrap class files
  -extdirs <dirs>   Override location of installed extensions
  -endorseddirs <dirs> Override location of endorsed standards path
  -proc:{none,only} Control whether annotation processing and/or compilation is done.
  -processor <class1>[,<class2>,<class3>...] Names of the annotation processors to run; bypasses default discovery process
  -processorpath <path> Specify where to find annotation processors
  -d <directory>     Specify where to place generated class files
  -s <directory>     Specify where to place generated source files
  -implicit:{none,class} Specify whether or not to generate class files for implicitly referenced files
  -encoding <encoding> Specify character encoding used by source files
  -source <release>  Provide source compatibility with specified release

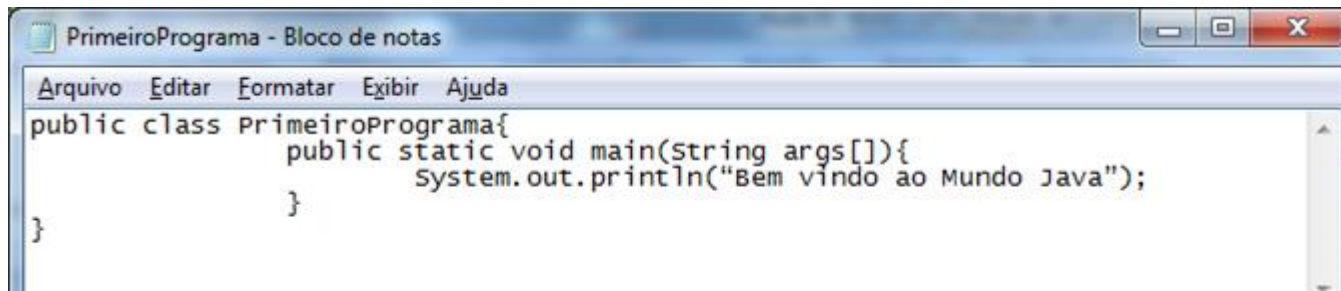
  -target <release>  Generate class files for specific VM version
  -version           Version information
  -help             Print a synopsis of standard options
  -Akey[=value]     Options to pass to annotation processors
  -X               Print a synopsis of nonstandard options
  -J<flag>          Pass <flag> directly to the runtime system

C:\Users\Eduardo>_
```

Escrevendo o primeiro programa

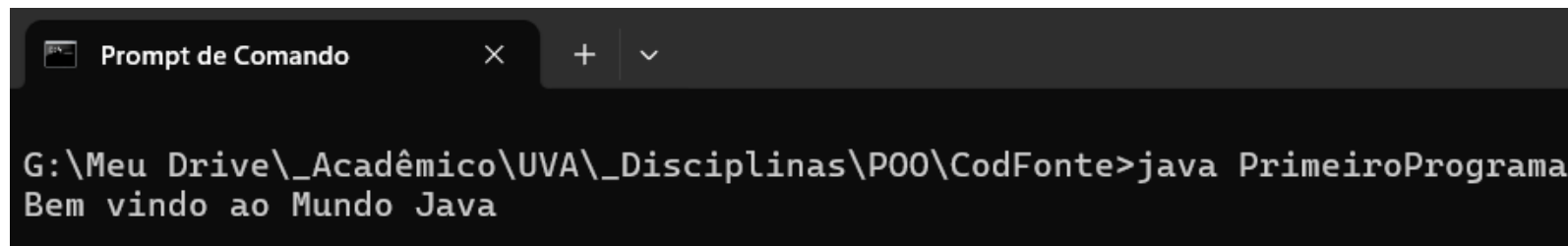
Montando Ambiente

- Com o Bloco de Notas, digitar o texto do PrimeiroPrograma.
 - Note que o arquivo deverá ser salvo com o mesmo nome da classe
- Gravar a classe com o mesmo nome do arquivo .java



```
Arquivo  Editar  Formatar  Exibir  Ajuda
public class PrimeiroPrograma{
    public static void main(String args[]){
        System.out.println("Bem vindo ao Mundo Java");
    }
}
```

- Compile o programa no terminal
- Execute o programa como mostra a figura abaixo



```
Prompt de Comando
G:\Meu Drive\_Acadêmico\UVA\_Disciplinas\P00\CodFonte>java PrimeiroPrograma
Bem vindo ao Mundo Java
```

Escrevendo o primeiro programa

Montando Ambiente

- Acabamos de executar nosso primeiro programa.
- Observação: O compilador java e a sua máquina virtual, foram instalados dentro da pasta 'bin' da pasta da versão do JDK escolhida. Para que você consiga chamar estes programas de qualquer parte do seu computador, você terá que incluir o caminho da pasta 'bin' na variável de ambiente do seu sistema chamada 'Path'.

As convenções do Java

Programação OO

- Java é “Case Sensitive”. As convenções utilizadas:
 - ✓ Nome de variáveis e métodos começam com letras minúsculas
 - ✓ Nome de classes iniciam com letras maiúsculas;
 - ✓ Nome composto: utilizar letras maiúsculas para as iniciais das palavras;
 - ✓ Letras maiúsculas para as constantes;

Case Sensitive – Em computação significa que um programa ou um compilador faz a diferenciação entre letras maiúsculas e minúsculas, ou seja, Maiúscula é diferente de maiúscula. O sistema operacional Linux é case sensitive bem como as linguagens C, Java, C# entre outras.

Comentários em Java

Programação OO

- Existem três formas de se inserir comentários :
 - ✓ // Comentário em uma linha
 - ✓ /* Comentário em uma ou mais linhas */
 - ✓ /** Documento Comentários */

- Quando o comentário tipo 3 é colocado imediatamente acima da declaração (de uma função ou variável), indica que o comentário poderá ser incluído automaticamente em uma página HTML (gerado pelo comando javadoc – gerador de documentação do Java).

Tipo de dados em Java

Programação OO



Tipo	Tamanho	Faixa de Valores
int	4 bytes	-2.147.483.648 até 2.147.483.647
short	2 bytes	-32.768 até 32.767
byte	1 bytes	-128 até 127
long	8 bytes	-9.223.372.036.854.775.808 até 9.223.372.036.854.775.807
float	4 bytes	+/- 3.40282347E+38F (aproximadamente 7 dígitos significativos)
double	8 bytes	+/- 1.79769313486231570E+308 (15 dígitos significativos)
char	2 bytes	0 até 65.536
boolean	1 bit	True ou false

Declarar e atribuir valores

Programação OO



- `int x, y; //declarando duas variáveis inteiras`
- `x = 6; //atribuindo valores a variáveis`
- `y = 1000;`
- `float f = 3,141516f; //ponto flutuante`
- `double w = 3,2310834; //ponto flutuante - dupla precisão`
- `char ch = 'a'; //Caractere`
- `final int MAX = 9; //Define a constante MAX com 9`

Operadores Aritméticos

Programação OO



Operador	Uso	Descrição
+	op1 + op2	Retorna a soma de op1 e op2
-	op1 - op2	Retorna a subtração de op1 por op2
*	op1 * op2	Retorna a multiplicação de op1 por op2
/	op1 / op2	Retorna a divisão de op1 por op2
%	op1 % op2	Retorna o resto da divisão de op1 por op2
++	++op	Incrementa op de 1, depois retorna o valor
++	op++	Retorna o valor de op, depois o incrementa de 1
--	--op	Decrementa op de 1, depois retorna o valor
--	op--	Retorna o valor de op, depois o decrementa de 1

Operadores Relacionais

Programação OO



Operador	Uso	Retorna verdadeiro se
>	op1>op2	op1 for maior que op2
>=	op1>=op2	op1 for maior ou igual que op2
<	op1<=op2	op1 for menor que op2
<=	op1<=op2	op1 for menor ou igual que op2
==	op1==op2	op1 for igual que op2
!=	op1!=op2	op1 for diferente que op2
&&	op1&&op2	Se op1 e op2 forem verdadeiros
	op1 op2	Se op1 ou op2 for verdadeiro

□ O operador ! é chamado de not ou negado



- Conjunto de linhas de códigos situadas entre um abre e um fecha chaves({ }). É permitido criar blocos dentro de blocos.

```
{ //início de bloco
```

```
...
```

```
    /*bloco de comandos*/
```

```
...
```

```
} //fim de bloco
```



- Escopo de uma variável indica em que parte do código ou bloco de comandos do programa que podemos utilizar ou enxergar a variável. Existem **variáveis locais** e **variáveis globais**. O escopo define também quando a variável será criada e destruída da memória. **As locais estão visíveis apenas dentro do bloco enquanto as globais estão disponíveis em qualquer bloco do programa.**

Observação: escopo é diferente de visibilidade, o qual se aplica as variáveis de classe e tem a ver com a utilização destas fora da classe.

Comando Condicional

Estruturas Básicas de Programação



- Desvia o fluxo natural do programa de acordo com o resultado de um teste lógico.

if (expressão booleana)

comando1 ou {bloco de comandos1}

else

comando2 ou {bloco de comandos2}

Comando Condicional (cont.)

Estruturas Básicas de Programação



- Quando existe um conjunto de opções, podemos utilizar a estrutura switch – case

switch(variável)

```
{  
    case(valor1):comando1; break;  
  
    case(valor2):comando2; break;  
  
    case(valor3):comando3; break;  
  
    ....  
  
    default:comando_genérico; break;  
  
}
```

Desvio de Fluxo

Estruturas Básicas de Programação



- Existem dois tipos de desvios de fluxo
 - ✓ Break
 - ✓ Continue;
- **break;**
 - O comando termina a execução de um loop sem executar o resto dos comando e **força a saída do laço.**
- **continue;**
 - O comando termina a execução de um laço sem executar o resto dos comandos, **voltando para o início do laço**, para uma nova iteração.

Exercícios da Semana 1

Exercícios Semana 1

1) Programa básico para escrever a famosa frase: “Olá Mundo!”



2) Programa para escrever a soma de dois números inteiros. Os números são: 2 e 8.

3) Programa para ler dois números do tipo double e escrever a soma deles.

4) Programa ler dois números do tipo float e informar o maior entre eles. Utilize a estrutura condicional “if”

5) Programa para ler o número referente ao dia da semana e, então, escrever o nome do dia da semana correspondente ao número informado.
Utilizar a função switch()

Exercícios Semana 1

1) Programa básico para escrever a famosa frase: "Olá Mundo!"



```
package semana1;

public class OlaMundo {

    public static void main(String[] args) {
        System.out.println("Olá Mundo!");
    }

}
```

Exercícios Semana 1

2) Programa para escrever a soma de dois números inteiros. Os números são: 2 e 8.



```
package semana1;  
public class SomaDoisNrs_v1 {  
    public static void main(String[] args) {  
        int num1, num2, soma;  
        num1=2;  
        num2=8;  
        soma = num1+num2;  
        System.out.println("A soma é "+ soma);  
    }  
}
```

Exercícios Semana 1

3) Programa para ler dois números do tipo double e escrever a soma deles.



```
package semana1;
import java.util.Scanner;
public class SomaDoisNrs_v2 {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        double n1, n2, soma;

        System.out.print("Primeiro número: ");
        n1 = entrada.nextDouble();

        System.out.print("Digite o segundo número: ");
        n2 = entrada.nextDouble();

        soma = n1 + n2;

        System.out.println("A soma dos dois números é: " + soma);
    }
}
```

Exercícios Semana 1

3) Programa para ler dois números do tipo double e escrever a soma deles.



```
package semana1;
import java.util.Scanner;
public class SomaDoisNrs_v2 {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);

        System.out.print("Primeiro número: ");
        double n1 = entrada.nextDouble();

        System.out.print("Digite o segundo número: ");
        double n2 = entrada.nextDouble();

        double soma = n1 + n2;

        System.out.println("A soma dos dois números é: " + soma);
    }
}
```

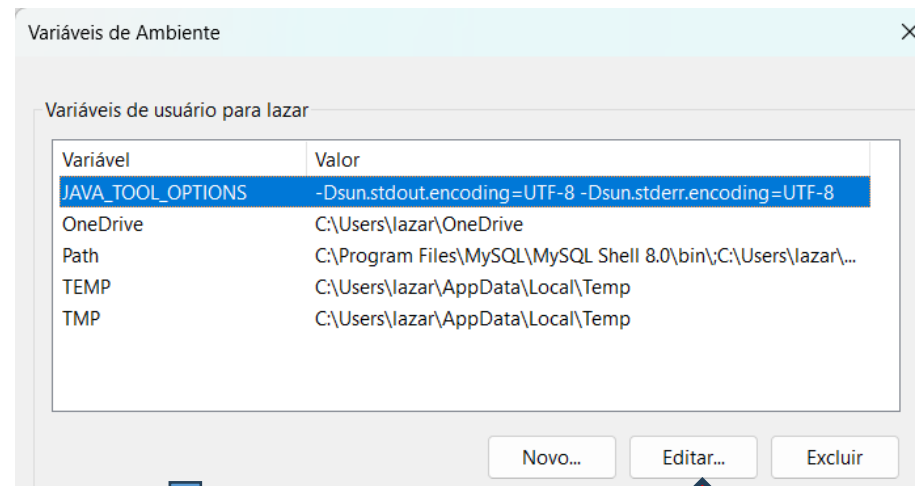
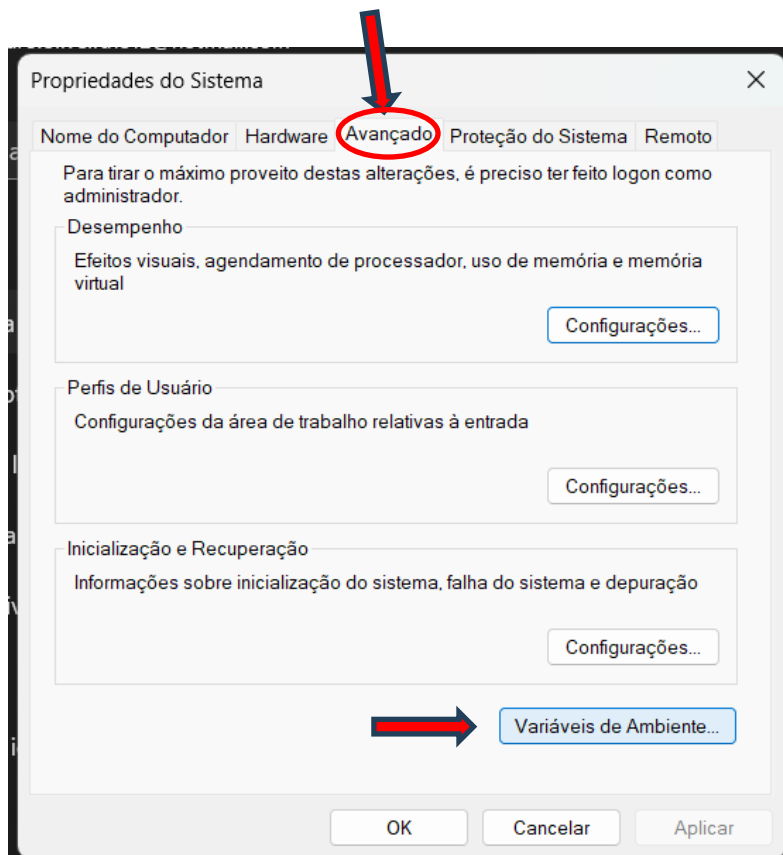
Variáveis criadas em tempo de execução

Exercícios Semana 1

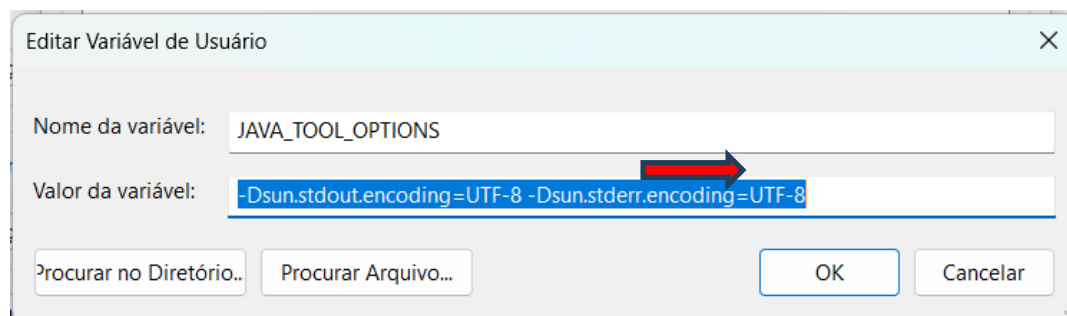
Para exibir os acentos, é preciso alterar (ou criar) a variável de ambiente de sistema, de nome:

“JAVA_TOOL_OPTIONS” e, nela, digitar o seguinte valor abaixo:

-Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8



Clique aqui, se já existir
Clique aqui, se não existir



Exercícios Semana 1

4) Programa ler dois números do tipo float e informar o maior entre eles. Utilize a estrutura condicional "if"



```
package semana1;
import java.util.Scanner;
public class MaiorNum {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        float n1, n2, maior;

        System.out.print("Primeiro número: ");
        n1 = entrada.nextFloat();

        System.out.print("Digite o segundo número: ");
        n2 = entrada.nextFloat();

        if (n1 > n2)
            maior = n1;
        else
            maior = n2;

        System.out.println("O maior número é: " + maior);
    }
}
```

Diagram illustrating the conditional logic for finding the maximum of two numbers:

```
if (n1 > n2) {
    maior = n1;
} else {
    maior = n2;
}
```

Exercícios Semana 1

5) Programa para ler o número referente ao dia da semana e, então, escrever o nome do dia da semana correspondente ao número informado. Utilizar a função switch()

```
package semana1;
import java.util.Scanner;
public class DiasSemana {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int dia;
        String nome_dia;

        System.out.print("Informe o número referente ao dia da semana (de 1 à 7): ");
        dia = entrada.nextInt();
```



```
switch(dia) {
    case 1:
        nome_dia = "Domingo";
        break;
    case 2:
        nome_dia = "Segunda-feira";
        break;
    case 3:
        nome_dia = "Terça-feira";
        break;
    case 4:
        nome_dia = "Quarta-feira";
        break;
    case 5:
        nome_dia = "Quinta-feira";
        break;
    case 6:
        nome_dia = "Sexta-feira";
        break;
    case 7:
        nome_dia = "Sábado";
        break;
    default:
        nome_dia = "Valor inválido. Digite um número entre 1 e 7!";
        break;
}
System.out.println("Dia da semana: " + nome_dia);
}
```